COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	10-1-2026	**DATE SUBMITTED:**	10-1-2026
NAME:	Canoy Hail B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 **Hands-on Activity 10.1 | Graphs**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list.

Code

ILOA > main.cpp > main()

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head;  // point new node to current head

        return newNode;
    }

    int N;  // number of nodes in the graph

public:
    adjNode** head;  // adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N];
        this->N = N;

        // initialize head pointers for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;

        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // insert in the beginning
            adjNode* newNode =
                getAdjListNode(end_ver, weight, head[start_ver]);

            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }

```

```cpp
    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {

            adjNode* current = head[i];

            while (current != nullptr) {
                adjNode* temp = current;
                current = current->next;

                // I replaced delete[] with delete because it was causing an error.
                // The nodes were created using new, so each node must use delete.
                delete temp;
            }
        }

        // I kept delete[] here because head was created as an array using new[].
        delete[] head;
    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
                  << ", " << ptr->cost << ") ";

        ptr = ptr->next;
    }

    std::cout << std::endl;
}

// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},
        {0,2,4},
        {1,4,3},
        {2,3,2},
        {3,1,4},
        {4,3,3}
    };

    int N = 6;  // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout << "Graph adjacency list "
              << std::endl
              << "(start_vertex, end_vertex, weight):"
              << std::endl;
```

```
118     for (int i = 0; i < N; i++)
119     {
120         // display adjacent vertices of vertex i
121         display_AdjList(diagraph.head[i], i);
122     }
123
124     return 0;
125 }
126
```

Output

```
PS C:\Users\Ian Canoy\Downloads\10.1HOA\ILOA\output> & .\'main.exe'
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Observations

- The error in this code is in the destructor. The delete[] head[i] is trying to delete the adjNode as an array, but the nodes were created one by one using new adjNode. So I replaced it with a while loop that deletes each node using delete. After that, the program can properly remove each node in the adjacency list without causing an error.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search.

B.1. Depth-First Search

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

// Step 2. Write the following struct in order to implement an edge in our graph:
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Step 3. Next, overload the << operator for the graph so that it can be printed out using the following function:
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }

    return os;
}

// Step 4. Implement the graph data structure that uses an edge list representation as follows:
template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }

    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }
```

```cpp
    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;

        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }

        return edges_from_v;
    }

    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

// Step 5. Now, we need a function to perform DFS for our graph. Implement it as follows:
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
```

```
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }

    return visit_order;
}

// Step 6. We shall test our implementation of the DFS on the graph shown here:
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

// Step 7. Finally, add the following test and driver code, which runs our DFS implementation and prints the output:
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();

    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;

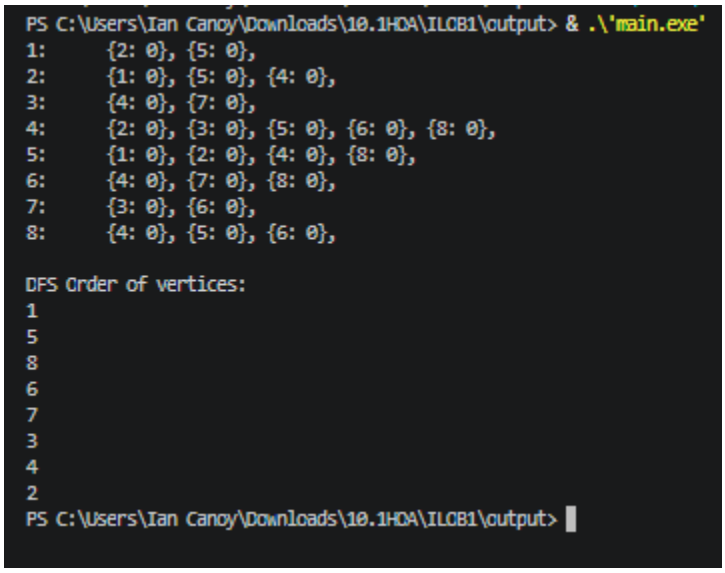
    auto dfs_visit_order = depth_first_search(G, 1);

    for (auto v : dfs_visit_order)
        // I replaced this because it was causing an error.
        std::cout << v << std::endl;
}
```

```
// Step 8. Compile and run the preceding code. Include your output as a table in section 6.
int main()
{
    using T = unsigned;

    test_DFS<T>();

    return 0;
}
```

Output	
Observations	- The error in this code is just in one line. The std::cout << v << std: was incomplete, so I replaced it with std::cout << v << std::endl;. After that, the program can properly print each vertex in the DFS order. The unused parameter 'dest' is only a warning, so I did not change it.

B.2. Breadth-First Search

Code

ILOB2 > main.cpp > main()

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

// Step 2: Write the following struct, which represents an edge in our graph:
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Step 3: Next, overload the << operator for the Graph data type in order to display the contents of the graph:
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }

    return os;
}

// Step 4: Write a class to define our graph data structure, as shown here:
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }
```

```
65      void add_edge(Edge<T> &&e)
66      {
67          if (e.src >= 1 && e.src <= V &&
68              e.dest >= 1 && e.dest <= V)
69              edge_list.emplace_back(e);
70          else
71              std::cerr << "Vertex out of bounds" << std::endl;
72      }
73
74      auto outgoing_edges(size_t v) const
75      {
76          std::vector<Edge<T>> edges_from_v;
77
78          for (auto &e : edge_list)
79          {
80              if (e.src == v)
81              {
82                  edges_from_v.emplace_back(e);
83              }
84          }
85
86          return edges_from_v;
87      }
88
89      // I replaced this because it was causing an error.
90      template <typename U>
91      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
92
93  private:
94      size_t V;
95      std::vector<Edge<T>> edge_list;
96  };
97
98  // Step 5: For this exercise, we shall test our implementation of BFS on the following graph:
99
100 // We need a function to create and return the required graph. Note that while edge weights are assigned to
101 // each edge in the graph, this is not necessary since the BFS algorithm does not need to use edge weights.
102 // Implement the function as follows:
103 template <typename T>
104 auto create_reference_graph()
105 {
106     Graph<T> G(9);
107
108     std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
109
110     edges[1] = {{2, 2}, {5, 3}};
111     edges[2] = {{1, 2}, {5, 5}, {4, 1}};
112     edges[3] = {{4, 2}, {7, 3}};
113     edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
114     edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
115     edges[6] = {{4, 4}, {7, 4}, {8, 1}};
116     edges[7] = {{3, 3}, {6, 4}};
117     edges[8] = {{4, 5}, {5, 3}, {6, 1}};
118
119     for (auto &i : edges)
120         for (auto &j : i.second)
121             G.add_edge(Edge<T>{i.first, j.first, j.second});
122
123     return G;
124 }
```

```cpp
// Step 6: Implement the breadth-first search like so:
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }

    return visit_order;
}

// Step 7: Add the following test and driver code that creates the reference graph, runs BFS starting from vertex
// 1, and outputs the results:
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();

    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;

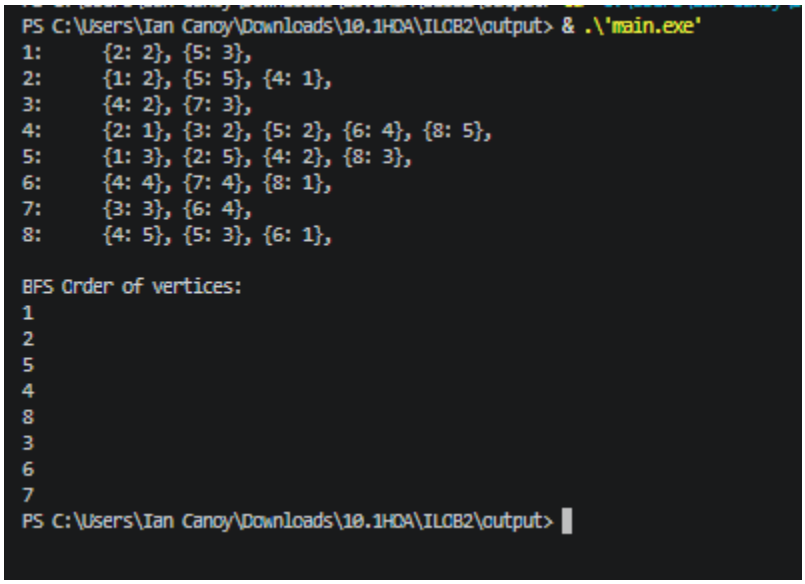
    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;

    test_BFS<T>();

    return 0;
}
```

Output	
Observations	- The error in this code is in the Graph class. The friend function was using template <typename T> again even though the class was already using T, which caused a template parameter shadowing error. I replaced the inner T with U. I also added return os; because the operator<< function needs to return the output stream. After these changes, the code should compile properly. The dest and signedness messages are only warnings.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

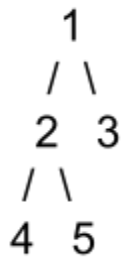
ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- The **Depth-First Search (DFS)** algorithm would be most helpful for this task. It starts from one vertex and explores as far as possible through one path before **backtracking** to the previous vertex and exploring another path. This matches the situation described in the question.

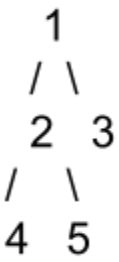
2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

DFS in Graph



$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3$

DFS in Tree



Preorder: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3$
Inorder: $4 \rightarrow 2 \rightarrow 5 \rightarrow 1 \rightarrow 3$
Postorder: $4 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 1$

```
DFS(vertex):
    mark vertex as visited
    for each adjacent vertex:
        if not visited:
            DFS(adjacent vertex)
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1);

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }

    return visit_order;
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?
   - A queue is used to implement the Breadth-First Search (BFS).

4. How many times can a node be visited in the BFS?
   - A node can be visited only once in the BFS because the code uses a visited set to check if the node has already been visited.

Conclusion
- In this activity I learned how graphs can be represented and how DFS and BFS work and their properties. It's easy to understand but when it comes to hard coding it manually it gets rough and very hard to understand although I understand how the code works it's still a little complicated for me. Overall I learned the differences of both DFS and BFS and how DFS uses stack and BFS uses queue.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **ChatGpt.**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Checking and explaining some concepts I don't know.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used it only as a guide.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.